

Artificial Intelligence Assignment Report 1 $\LaTeX$

Warren Roberts

Student ID: 100450116

Student Email: wlr006@uark.edu

University of Arkansas

0 Days late used

Abstract

This report is designed to serve as documentation and reflection on the work required and completed in Assignment 1 of Artificial Intelligence CSCE-46103. Below will go through the steps of designing algorithms, where related information was gathered, and my line of thought during the process of reaching completeness.

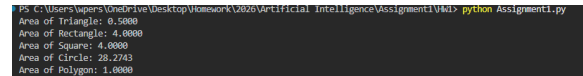
Introduction

The goal of the assignment was to implement fundamental AI-related algorithms using appropriate data structures while maintaining correctness and efficiency. To begin, assignment one required me to use multiple Python 3 libraries, such as `heapq`, `deque`, and `numpy`. These libraries offer documentation covering their methods, variables, and syntax. From a starting point, I was given source code that contained skeletons of classes, methods, and test methods. With these, building methods became much simpler, as their architecture was already provided. Resources regarding the data structures used within the project have been linked in the reference section below. These sites helped me understand the basics and implementation steps of each data type. Each one provides a different way to structure and manipulate data, with some proving more helpful for specific tasks.

Class and Inheritance

The first portion of the assignment focused on object-oriented design through the implementation of several geometric shapes using class inheritance. Using class inheritance reminded me of polymorphism and other things I learned in programming paradigms. A base `Geometry` class was provided, which stored shared attributes such as a name and a list of points (working as a starting point for all other classes). From this base class, specific shapes including `Triangle`, `Rectangle`, `Square`, `Circle`, and `Polygon` were derived. Each subclass implemented its own version of the `calculate_area()` method while inheriting common behavior from the parent class (`Geometry`).

This design allowed functionality to be reused while still implementing special cases and behaviors as needed. For example, `Square` inherits from `Rectangle` since both rely



```
PS C:\Users\wlr006\OneDrive\Desktop\Homework\2020\Artificial Intelligence\Assignment1\MD> python Assignment1.py
Area of Triangle: 0.5000
Area of Rectangle: 4.0000
Area of Square: 4.0000
Area of Circle: 28.2743
Area of Polygon: 1.0000
```

Figure 1: Output of geometric shape area calculations.

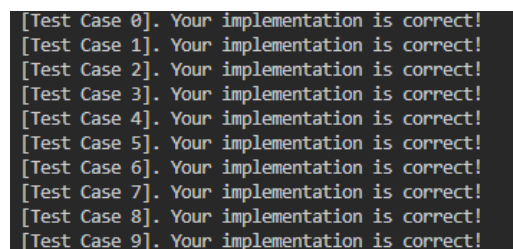
on axis-aligned vertices, but the square enforces equal side lengths. Similarly, `Polygon` generalizes area computation using the shoelace formula, allowing it to support any number of vertices. Using inheritance reduced redundancy in the code and made the structure of the program easier to understand and extend. With inheritance in-use, writing the code was much faster, with the definition of each subclass taking less space than it would without inheritance.

Matrix Multiplication

Matrix multiplication was implemented manually using three nested loops rather than relying on built-in NumPy functions. Given matrices A of size $(n \times k)$ and B of size $(k \times m)$, the algorithm computes the resulting matrix C of size $(n \times m)$ by iterating over each row of A and each column of B . Each entry in C is calculated as the dot product of the corresponding row and column vectors.

The implemented algorithm runs in $O(n \cdot m \cdot k)$ time due to the three nested loops. While this approach is less efficient than optimized library implementations, it provides a clear understanding of how matrix multiplication works at a fundamental level. A dimension check is performed before computation to ensure the matrices are compatible, preventing invalid operations.

This matrix multiplication function was later reused in



```
[Test Case 0]. Your implementation is correct!
[Test Case 1]. Your implementation is correct!
[Test Case 2]. Your implementation is correct!
[Test Case 3]. Your implementation is correct!
[Test Case 4]. Your implementation is correct!
[Test Case 5]. Your implementation is correct!
[Test Case 6]. Your implementation is correct!
[Test Case 7]. Your implementation is correct!
[Test Case 8]. Your implementation is correct!
[Test Case 9]. Your implementation is correct!
```

Figure 2: Output of `test_matrix_mul()` for matrix multiplication.

```

Iterative: A^4 = [[ [-1.54866293 -2.82837614 2.89559067 3.68538524]
[-0.06080728 1.08519784 -9.85736717 -12.59945459]
[ 2.10463716 -5.48289992 15.2378548 24.05308083]
[-0.86491192 -1.689297 4.07625922 4.89063675]]]
Iterative: A^5 = [[ [89.548880283 -138.06423525 5.78248864 -27.58286757 223.96173175]
[-20.98451872 22.68079204 -13.85349752 5.75087781 -45.31968275]
[ 56.62012731 -69.86888048 2.58774622 -6.41009363 110.8817653 ]
[-222.88659662 198.2678221 -29.31344591 31.22274951 -299.05754683]
[-51.57112875 30.83794928 -10.97636468 -13.31619115 -52.17346397]]]
Iterative: A^6 = [[ [-0.63139891 7.674823 5.4621635 -4.48790357]
[-0.07569011 -1.5675593 -1.11638188 3.78771745]
[ 2.57953312 1.79745595 -3.99040499 -0.54802383]
[ 1.23164495 -0.53110549 -2.74277949 -6.81274481]]]
Iterative: A^2 = [[ [-0.25766817 0.96348878]
[-1.72244907 1.20589219]]]
Iterative: A^5 = [[ [-4.20981236 0.10214477 0.66770871 -2.93841017 4.78889931]
[ 2.47872898 -31.2553978 -3.72192671 7.26134932 -3.45448521]
[ 9.76489718 -5.14007226 2.77195874 2.31626532 -13.63409099]
[-77.14916386 -32.33978007 -38.29681623 1.47530667 91.38759604]
[ 34.45977288 27.61843173 19.22509967 -3.42551225 -41.50809759]]]
Iterative: A^3 = [[ [7.65145563 13.06030148 3.97410443]
[ 9.14248796 11.52133783 9.24093881]
[ 2.30047345 2.68284273 0.4496072 ]
[ 1.63697048 -0.91956578 0.47481262]
[ 1.71888257 -0.15921015 -0.21655075]
[-2.64558972 -0.58833113 0.91899532]]]
Iterative: A^4 = [[ [1.05025595 -1.39995936 4.5052663 5.33281932]
[-1.8896199 5.55856915 -5.51525395 -8.24551598]
[-8.08677278 18.55424126 8.33524849 -6.23525174]
[-3.48764368 8.88064688 2.23377788 -3.59149113]]]
Iterative: A^4 = [[ [-0.50328029 0.04309106 -0.22126633 4.71468866]
[-0.30884592 -0.23693997 -0.71026389 1.54420288]
[ 1.39215616 -1.38298006 -0.92012168 -1.43283467]
[ 0.41004951 0.0700594 0.6150662 -0.65359359]]]
Iterative: A^2 = [[ [-0.01180341 0.03427695]
[-0.41778509 0.0168767 ]]]

```

Figure 3: Output of test_pow() for iterative power testing.

other parts of the assignment, including recursive and iterative matrix exponentiation. Reusing this implementation reinforced the importance of correctness, as any error in matrix multiplication would propagate into later computations such as Fibonacci number generation and general recursive sequences.

BFS, DFS, and findMinimum

Breadth-first search (BFS) and depth-first search (DFS) were implemented to traverse a grid-based environment represented as a two-dimensional NumPy array. Each cell in the grid could either be traversable or blocked. BFS was implemented using a queue, ensuring that the shortest path in terms of number of steps was found first. A parent mapping was maintained to reconstruct and output the path once the goal cell was reached.

DFS was implemented using an explicit stack to explore paths deeply before backtracking. While DFS does not guarantee the shortest path, it is useful for determining reachability and exploring all possible paths. A visited matrix was used in both algorithms to prevent revisiting cells and entering infinite loops.

The findMinimum function extends these ideas by incorporating path cost using a priority queue implemented with Python's heapq module. This algorithm is equivalent to Dijkstra's algorithm, where each cell's value represents the cost of entering that cell. The algorithm maintains a distance matrix and always expands the lowest-cost path next. This ensures that the minimum-cost path to the destination is found when one exists.

Conclusion

Overall, this assignment was a great way to start this course, due to each part interacting with the others. This meant that each implementation had to be correct before moving on to the next section. While this can seem daunting, it boosts confidence that your implementations are correct when the later

```

Recursive: A^4 = [[ [2.31864234 -0.34796213 0.39215513 -0.43498086]
[-2.58306158 0.21896753 -0.61562592 0.4214797 ]
[-4.93475864 2.24628685 0.25840109 2.14823251]
[-10.22863177 3.53285687 -0.70489254 4.11222116]]]
Recursive: A^4 = [[ [19.64535731 5.02971293 -4.26780627 2.93871121]
[ 5.65003688 -0.81119057 3.78479362 -4.01317507]
[ 1.71955601 -8.35046392 15.0683882 2.13241443]
[-6.65704554 -0.21334662 10.45181901 -3.60383704]]]
Recursive: A^3 = [[ [0.48590864 -0.33775957 -0.73785991]
[ 0.09936501 -0.25940476 0.28993288]
[ 1.53093191 -0.25474607 -1.41856864]]]
Recursive: A^2 = [[ [1.13106066 0.63333949]
[0.4535953 0.28341544]]]
Recursive: A^2 = [[ [-0.03887435 3.61597847]
[-0.73406659 3.61535985]]]
Recursive: A^4 = [[ [4.36877643 2.60833605 2.72511163 5.9912077 ]
[ 1.58025496 2.11880642 0.28752736 4.36685718]
[-1.16578477 -2.56692633 0.62944833 -5.34729044]
[ 3.07462532 0.0951184 2.99280374 0.88161453]]]
Recursive: A^5 = [[ [-4.59302425 51.68218575 -1.25315679 17.46446251 17.0976497 ]
[-7.18324137 -12.62417979 -22.75706457 -34.93266755 7.42889041]
[ 2.49333392 -0.68546301 -2.6706339 -5.93181345 0.57523218]
[ 0.64794597 88.45792073 -41.03677695 -35.14915097 43.23008043]
[ 7.8377859 3.56693591 -11.92600838 -23.81512582 8.6387668 ]]]]
Recursive: A^4 = [[ [2.2792298 0.8385585 4.32707574 1.60577567]
[ 2.53516987 0.23529766 3.81311159 0.25244784]
[ 1.28374356 1.6533759 2.55028381 1.91178238]
[ 0.43347781 0.29128838 0.24152577 -0.17521441]]]
Recursive: A^3 = [[ [2.49827121 2.73799625 -1.42381722]
[ 1.25648658 5.30808608 -1.19421628]
[-5.94285281 -7.66858416 3.52982716]]]
Recursive: A^5 = [[ [3.38199051e+00 -4.47100337e+01 -1.40818865e+01 3.16136984e+01
8.90316328e+00]
[ 4.28459127e+00 -4.18873546e+01 -1.51746252e+01 3.59733635e+01
2.21689066e+01]
[ 1.50954177e+00 1.06776498e+01 1.48978526e+00 -5.73058386e+00
-7.27562458e+00]
[ 3.52847258e-02 2.85453801e+01 5.64860774e+00 -1.81618450e+01
-1.72331955e+00]
[ 9.08127921e-01 -1.14288491e+01 -3.0098913e+00 1.18284640e+01
6.06827691e+00]]]

```

Figure 4: Output of test_pow() for recursive power testing.

```

Recursive: A^4 = [[ [2.31864234 -0.34796213 0.39215513 -0.43498086]
[-2.58306158 0.21896753 -0.61562592 0.4214797 ]
[-4.93475864 2.24628685 0.25840109 2.14823251]
[-10.22863177 3.53285687 -0.70489254 4.11222116]]]
Recursive: A^4 = [[ [19.64535731 5.02971293 -4.26780627 2.93871121]
[ 5.65003688 -0.81119057 3.78479362 -4.01317507]
[ 1.71955601 -8.35046392 15.0683882 2.13241443]
[-6.65704554 -0.21334662 10.45181901 -3.60383704]]]
Recursive: A^3 = [[ [0.48590864 -0.33775957 -0.73785991]
[ 0.09936501 -0.25940476 0.28993288]
[ 1.53093191 -0.25474607 -1.41856864]]]
Recursive: A^2 = [[ [1.13106066 0.63333949]
[0.4535953 0.28341544]]]
Recursive: A^2 = [[ [-0.03887435 3.61597847]
[-0.73406659 3.61535985]]]
Recursive: A^4 = [[ [4.36877643 2.60833605 2.72511163 5.9912077 ]
[ 1.58025496 2.11880642 0.28752736 4.36685718]
[-1.16578477 -2.56692633 0.62944833 -5.34729044]
[ 3.07462532 0.0951184 2.99280374 0.88161453]]]
Recursive: A^5 = [[ [-4.59302425 51.68218575 -1.25315679 17.46446251 17.0976497 ]
[-7.18324137 -12.62417979 -22.75706457 -34.93266755 7.42889041]
[ 2.49333392 -0.68546301 -2.6706339 -5.93181345 0.57523218]
[ 0.64794597 88.45792073 -41.03677695 -35.14915097 43.23008043]
[ 7.8377859 3.56693591 -11.92600838 -23.81512582 8.6387668 ]]]]
Recursive: A^4 = [[ [2.2792298 0.8385585 4.32707574 1.60577567]
[ 2.53516987 0.23529766 3.81311159 0.25244784]
[ 1.28374356 1.6533759 2.55028381 1.91178238]
[ 0.43347781 0.29128838 0.24152577 -0.17521441]]]
Recursive: A^3 = [[ [2.49827121 2.73799625 -1.42381722]
[ 1.25648658 5.30808608 -1.19421628]
[-5.94285281 -7.66858416 3.52982716]]]
Recursive: A^5 = [[ [3.38199051e+00 -4.47100337e+01 -1.40818865e+01 3.16136984e+01
8.90316328e+00]
[ 4.28459127e+00 -4.18873546e+01 -1.51746252e+01 3.59733635e+01
2.21689066e+01]
[ 1.50954177e+00 1.06776498e+01 1.48978526e+00 -5.73058386e+00
-7.27562458e+00]
[ 3.52847258e-02 2.85453801e+01 5.64860774e+00 -1.81618450e+01
-1.72331955e+00]
[ 9.08127921e-01 -1.14288491e+01 -3.0098913e+00 1.18284640e+01
6.06827691e+00]]]

```

Figure 5: Output from test_fibonacci() and test_bfs_dfs_find_minimum(), confirming correct Fibonacci computation and successful BFS/DFS/minimum-cost path results.

steps are compiling and outputting the expected results. I learned a lot about implementing stacks, queues, and heaps as data structures to be used in the manipulation of information. These implementations allow you to better understand the data at hand, giving you the knowledge to become a more skilled programmer. The methods and data structures implemented will bridge all topics within the broad field of

computer science, especially in artificial intelligence.

References

Queues in Python 3.

<https://www.geeksforgeeks.org/python/queue-in-python/>

Stacks in Python 3.

<https://www.geeksforgeeks.org/python/stack-in-python/>

Heapq in Python 3.

<https://docs.python.org/3/library/heapq.html>

Numpy Basics in Python 3.

<https://numpy.org/doc/2.4/user/basics.creation.html>